

BCA

IV SEMESTER

INSPIRE • LEARN • LEAD

ACCORDING TO
LATEST
SYLLABUS



JAI NARAYAN VYAS
UNIVERSITY, JODHPUR

OPERATING SYSTEM

A COMPLETE TEXTBOOK FOR BCA STUDENTS

COVERS



Process Management



Synchronization & Deadlock



Memory Management



Virtual Memory



Secondary Storage Management



Mobile & Linux Operating Systems



LEARN TODAY, LEAD TOMORROW

1. Operating System (OS) की परिभाषा

परिचय

कंप्यूटर सिस्टम मुख्य रूप से दो भागों से मिलकर बना होता है:

1. **Hardware (हार्डवेयर)** – जैसे CPU, RAM, हार्ड डिस्क, कीबोर्ड आदि।
2. **Software (सॉफ्टवेयर)** – जैसे ऑपरेटिंग सिस्टम और एप्लिकेशन सॉफ्टवेयर।

हार्डवेयर स्वयं कार्य नहीं कर सकता। उसे नियंत्रित करने और उपयोगकर्ता के निर्देशों को हार्डवेयर तक पहुँचाने के लिए एक विशेष सॉफ्टवेयर की आवश्यकता होती है जिसे **Operating System (OS)** कहा जाता है।

Operating System क्या है?

Operating System एक System Software है जो User और Computer Hardware के बीच मध्यस्थ (Interface) के रूप में कार्य करता है तथा कंप्यूटर के सभी संसाधनों का प्रबंधन और नियंत्रण करता है।

यह उपयोगकर्ता द्वारा दिए गए आदेशों को समझता है, उन्हें हार्डवेयर तक पहुँचाता है और प्राप्त परिणामों को उपयोगकर्ता तक वापस पहुँचाता है।

विद्वानों द्वारा परिभाषाएँ

1. Resource Manager के रूप में

Operating System कंप्यूटर के सभी संसाधनों जैसे CPU, Memory, Disk तथा Input-Output Devices का प्रबंधन करता है।

2. Control Program के रूप में

यह विभिन्न प्रोग्रामों के निष्पादन को नियंत्रित करता है तथा सिस्टम में त्रुटियों और संसाधनों के दुरुपयोग को रोकता है।

3. Interface के रूप में

यह उपयोगकर्ता और कंप्यूटर हार्डवेयर के बीच संचार स्थापित करता है।

Operating System के उदाहरण

- Microsoft Windows
- Linux
- macOS
- Android
- Ubuntu

Operating System का मुख्य उद्देश्य

1. कंप्यूटर का उपयोग आसान बनाना।
2. हार्डवेयर संसाधनों का कुशल उपयोग करना।
3. विभिन्न प्रोग्रामों के बीच समन्वय स्थापित करना।
4. डेटा और सिस्टम की सुरक्षा सुनिश्चित करना।
5. उपयोगकर्ता को सुविधाजनक वातावरण उपलब्ध कराना।

Operating System की आवश्यकता

यदि Operating System न हो तो:

- उपयोगकर्ता सीधे हार्डवेयर से संपर्क नहीं कर पाएगा।

- प्रत्येक प्रोग्राम को हार्डवेयर नियंत्रित करने के लिए अलग कोड लिखना पड़ेगा।
- संसाधनों का प्रबंधन कठिन हो जाएगा।
- कंप्यूटर का उपयोग सामान्य उपयोगकर्ताओं के लिए लगभग असंभव हो जाएगा।

Operating System का कार्य करने का तरीका

User



Application Software



Operating System



Hardware

जब उपयोगकर्ता कोई कार्य करता है, जैसे किसी फाइल को खोलना, तो एप्लिकेशन उस अनुरोध को Operating System तक भेजती है। Operating System आवश्यक हार्डवेयर संसाधनों का उपयोग करके कार्य पूरा करता है और परिणाम उपयोगकर्ता को वापस देता है।

2. Operating System के प्रकार (Types of Operating System)

कंप्यूटर तकनीक के विकास के साथ विभिन्न प्रकार के Operating System विकसित हुए। प्रत्येक प्रकार विशेष आवश्यकताओं को पूरा करने के लिए बनाया गया है।

2.1 Batch Operating System

परिचय

यह Operating System का सबसे प्रारम्भिक रूप था। इसमें उपयोगकर्ता सीधे कंप्यूटर से संपर्क नहीं करता था। समान प्रकार के कार्यों को एक समूह (Batch) में एकत्रित करके निष्पादित किया जाता था।

उदाहरण के लिए, यदि 100 छात्रों के परीक्षा परिणाम तैयार करने हों, तो सभी डेटा को एक साथ प्रोसेस किया जाएगा।

कार्यप्रणाली

User Jobs



Batch Formation



Operating System



Execution

विशेषताएँ

- User Interaction नहीं होता।
- समान कार्यों को समूह में निष्पादित किया जाता है।
- CPU का उपयोग बेहतर होता है।

लाभ

- बड़े कार्यों के लिए उपयुक्त।
- CPU Idle Time कम होता है।

- प्रबंधन आसान होता है।

हानियाँ

- परिणाम प्राप्त करने में अधिक समय लगता है।
- त्रुटि मिलने पर पूरा Batch दोबारा चलाना पड़ सकता है।
- User को तुरंत प्रतिक्रिया नहीं मिलती।

2.2 Multiprogramming Operating System

इस प्रणाली में एक समय में मुख्य स्मृति में कई प्रोग्राम रखे जाते हैं।

जब एक प्रोग्राम Input/Output कार्य के कारण प्रतीक्षा कर रहा होता है, तब CPU दूसरे प्रोग्राम को निष्पादित करता है। इससे CPU का उपयोग अधिकतम होता है।

उदाहरण

मान लीजिए तीन प्रोग्राम हैं:

- Program A प्रिंटर का उपयोग कर रहा है।
- Program B डिस्क से डेटा पढ़ रहा है।
- Program C CPU पर गणना कर रहा है।

यदि Program A और B प्रतीक्षा कर रहे हैं, तो CPU Program C को निष्पादित करेगा।

लाभ

- CPU Utilization बढ़ती है।
- Throughput बढ़ता है।
- सिस्टम की कार्यक्षमता बढ़ती है।

हानियाँ

- Memory Management जटिल होती है।
- Scheduling की आवश्यकता होती है।

2.3 Multitasking Operating System

जब एक उपयोगकर्ता एक ही समय में कई कार्य कर सकता है, तो उसे Multitasking कहा जाता है।

उदाहरण:

- इंटरनेट ब्राउज़ करना।
- संगीत सुनना।
- डॉक्यूमेंट तैयार करना।
- वीडियो डाउनलोड करना।

ये सभी कार्य एक साथ किए जा सकते हैं।

प्रकार

1. Preemptive Multitasking

Operating System आवश्यकता पड़ने पर CPU को एक Process से दूसरी Process को दे सकता है।

2. Cooperative Multitasking

Process स्वयं CPU छोड़ती है।

लाभ

- उपयोगकर्ता की उत्पादकता बढ़ती है।
- समय की बचत होती है।

2.4 Time Sharing Operating System

इस प्रकार के Operating System में CPU का समय छोटे-छोटे भागों में विभाजित किया जाता है जिन्हें **Time Slice** कहते हैं।

प्रत्येक उपयोगकर्ता को CPU का थोड़ा-थोड़ा समय दिया जाता है।

उदाहरण

यदि 10 उपयोगकर्ता एक सिस्टम का उपयोग कर रहे हैं, तो Operating System प्रत्येक उपयोगकर्ता को कुछ मिलीसेकंड का CPU समय देगा।

लाभ

- Response Time कम होता है।
- कई उपयोगकर्ता एक साथ कार्य कर सकते हैं।

हानियाँ

- Security की समस्या हो सकती है।
- Memory की अधिक आवश्यकता होती है।

2.5 Multiprocessing Operating System

इसमें एक से अधिक CPU या Processor होते हैं।

एक ही समय में कई कार्यों को अलग-अलग Processor द्वारा निष्पादित किया जाता है।

प्रकार

Symmetric Multiprocessing (SMP)

सभी Processor समान कार्य करते हैं।

Asymmetric Multiprocessing (AMP)

एक Processor नियंत्रक की भूमिका निभाता है।

लाभ

- गति बढ़ती है।
- विश्वसनीयता बढ़ती है।
- बड़ी गणनाओं के लिए उपयोगी।

2.6 Distributed Operating System

इस प्रणाली में कई कंप्यूटर नेटवर्क के माध्यम से जुड़े होते हैं और उपयोगकर्ता को एक ही सिस्टम की तरह दिखाई देते हैं।

विशेषताएँ

- Resource Sharing
- Load Sharing
- High Reliability

उपयोग

- वैज्ञानिक अनुसंधान
- क्लाउड कंप्यूटिंग
- विश्वविद्यालय नेटवर्क

2.7 Network Operating System

यह नेटवर्क में जुड़े कंप्यूटरों के प्रबंधन के लिए उपयोग किया जाता है।

कार्य

- User Authentication
- File Sharing
- Printer Sharing
- Network Security

2.8 Real Time Operating System (RTOS)

ऐसे Operating System जिनमें कार्य को निश्चित समय सीमा के भीतर पूरा करना अनिवार्य होता है।

उदाहरण

- एयर ट्रैफिक कंट्रोल सिस्टम
- मेडिकल उपकरण
- औद्योगिक रोबोट

प्रकार

Hard Real Time System

समय सीमा का पालन अनिवार्य है।

उदाहरण:

- मिसाइल नियंत्रण प्रणाली।

Soft Real Time System

समय सीमा महत्वपूर्ण है लेकिन थोड़ी देरी स्वीकार्य है।

उदाहरण:

- वीडियो कॉन्फ्रेंसिंग।

3. Functionalities and Characteristics of Operating System

3.1 Process Management

Process किसी Program का Running Instance होता है।

Operating System:

- Process बनाता है।
- Process समाप्त करता है।
- CPU आवंटित करता है।
- Synchronization करता है।

3.2 Memory Management

Operating System तय करता है:

- कौन-सा Process कितनी Memory उपयोग करेगा।
- Memory कब आवंटित और मुक्त होगी।
- Virtual Memory का उपयोग कैसे होगा।

3.3 File Management

Operating System:

- File बनाता है।
- File हटाता है।
- File का नाम बदलता है।
- Access अधिकार निर्धारित करता है।

3.4 Device Management

Operating System सभी Input और Output उपकरणों को नियंत्रित करता है।

उदाहरण:

- Printer
- Keyboard
- Mouse
- Scanner

3.5 Security and Protection

Operating System:

- Password Protection देता है।
- Unauthorized Access रोकता है।
- Data Encryption प्रदान करता है।

3.6 User Interface

CLI (Command Line Interface)

उपयोगकर्ता कमांड लिखकर कार्य करता है।

GUI (Graphical User Interface)

उपयोगकर्ता आइकन और विंडो का उपयोग करता है।

3.7 Error Detection

Operating System:

- Memory Errors पहचानता है।
- Hardware Failure की सूचना देता है।
- Disk Errors का पता लगाता है।

4. Hardware Concepts Related to Operating System

CPU (Central Processing Unit)

CPU कंप्यूटर का मस्तिष्क है।

CPU के मुख्य भाग

ALU

गणितीय और तार्किक कार्य करता है।

Control Unit

सभी कार्यों का नियंत्रण करती है।

Registers

अत्यधिक तेज स्मृति।

Main Memory

RAM

अस्थायी स्मृति।

ROM

स्थायी स्मृति।

Cache Memory

CPU और RAM के बीच स्थित उच्च गति वाली स्मृति।

Secondary Storage

- Hard Disk
- SSD
- DVD
- Pen Drive

Interrupt

जब कोई डिवाइस CPU का ध्यान आकर्षित करती है तो उसे Interrupt कहते हैं।

DMA

Direct Memory Access में I/O Device सीधे Memory से डेटा का आदान-प्रदान कर सकती है।

5. Operating System Services and System Calls

OS Services

1. Program Execution
2. I/O Operations
3. File Manipulation
4. Communication
5. Error Detection
6. Resource Allocation
7. Accounting
8. Protection and Security

System Call

System Call वह तंत्र है जिसके माध्यम से User Program Operating System से सेवाएँ प्राप्त करता है।

उदाहरण

जब कोई प्रोग्राम फाइल खोलना चाहता है:

Application Program



System Call (Open File)



Operating System



Disk Access

System Call के प्रकार

Process Control

- Create Process
- End Process

File Management

- Open
- Read
- Write
- Close

Device Management

- Request Device
- Release Device

Information Maintenance

- Get Time
- Set Date

Communication

- Send Message
- Receive Message

Protection

- Access Permission

6. System Programs

System Programs उपयोगकर्ता को Operating System के साथ कार्य करने में सहायता करते हैं।

उदाहरण

File Utilities

Copy, Move, Rename

Programming Support

Compiler, Interpreter, Assembler

Execution Support

Loader, Linker

Communication Programs

Browser, Email Client

Background Services

Antivirus, Printing Service

7. Operating System Structure

7.1 Monolithic Structure

पूरा Operating System एक ही बड़े Program के रूप में कार्य करता है।

7.2 Layered Structure

Operating System को कई परतों में विभाजित किया जाता है।

Layer 5 → User Interface

Layer 4 → File System

Layer 3 → Device Management

Layer 2 → Memory Management

Layer 1 → CPU Scheduling

Layer 0 → Hardware

7.3 Microkernel Structure

केवल आवश्यक सेवाएँ Kernel में रहती हैं।

7.4 Modular Structure

Operating System को छोटे-छोटे Modules में विभाजित किया जाता है।

7.5 Hybrid Structure

यह Monolithic और Microkernel दोनों का मिश्रण है।

उदाहरण:

- Microsoft Windows
- macOS

UNIT – II

Process Management and Synchronization

इस यूनिट में हम ऑपरेटिंग सिस्टम के सबसे महत्वपूर्ण भाग **Process Management** का अध्ययन करेंगे। किसी भी कंप्यूटर सिस्टम में जब कोई प्रोग्राम चलाया जाता है, तो वह केवल एक फाइल नहीं रहता बल्कि एक **Process** बन जाता है। ऑपरेटिंग सिस्टम का मुख्य कार्य इन Processes का प्रबंधन करना, CPU का उचित वितरण करना तथा विभिन्न Processes के बीच समन्वय स्थापित करना होता है।

1. Process Concepts (प्रोसेस की अवधारणा)

Process क्या है?

जब किसी प्रोग्राम को निष्पादन (Execution) के लिए मेमोरी में लोड किया जाता है और वह चलना शुरू करता है, तो उसे **Process** कहा जाता है।

अर्थात्,

A Process is a Program in Execution.

प्रोसेस वह प्रोग्राम है जो वर्तमान में चल रहा होता है।

Program और Process में अंतर

Program	Process
Program निर्देशों (Instructions) का संग्रह होता है।	Process उन निर्देशों का निष्पादन है।
यह हार्ड डिस्क में संग्रहित रहता है।	यह RAM में उपस्थित होता है।
Program निष्क्रिय (Passive) होता है।	Process सक्रिय (Active) होती है।
Program स्वयं CPU का उपयोग नहीं करता।	Process CPU तथा अन्य संसाधनों का उपयोग करती है।

उदाहरण:

यदि आपके कंप्यूटर में **MS Word** इंस्टॉल है तो वह एक **Program** है। लेकिन जैसे ही आप उसे खोलते हैं और उसमें कार्य करते हैं, वह एक **Process** बन जाता है।

Process के मुख्य घटक

एक Process मुख्य रूप से निम्न भागों से मिलकर बनी होती है:

1. Text Section (Code Segment)

इसमें प्रोग्राम का वास्तविक कोड होता है।

2. Data Section

इसमें Global Variables और Static Variables संग्रहित रहती हैं।

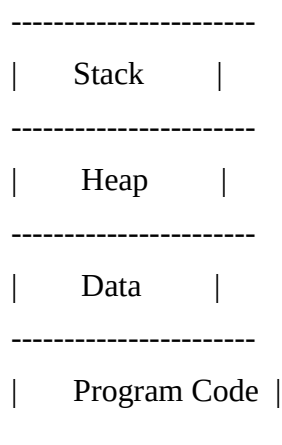
3. Heap Section

यह Dynamic Memory Allocation के लिए उपयोग होती है।

4. Stack Section

इसमें Function Calls, Local Variables और Return Addresses संग्रहित रहते हैं।

Process का मेमोरी लेआउट



Process के प्रकार

1. CPU Bound Process

ऐसी Process जो CPU का अधिक उपयोग करती है।

उदाहरण:

- गणितीय गणनाएँ
- वीडियो एन्कोडिंग

2. I/O Bound Process

ऐसी Process जो Input/Output Devices का अधिक उपयोग करती है।

उदाहरण:

- File Copying
- Printing

2. Process State and Process Control Block (PCB)

Process State (प्रोसेस की अवस्थाएँ)

एक Process अपने जीवनकाल में विभिन्न अवस्थाओं से गुजरती है।

1. New State

जब Process का निर्माण होता है।

2. Ready State

Process CPU प्राप्त करने के लिए तैयार होती है।

3. Running State

Process वर्तमान में CPU पर निष्पादित हो रही होती है।

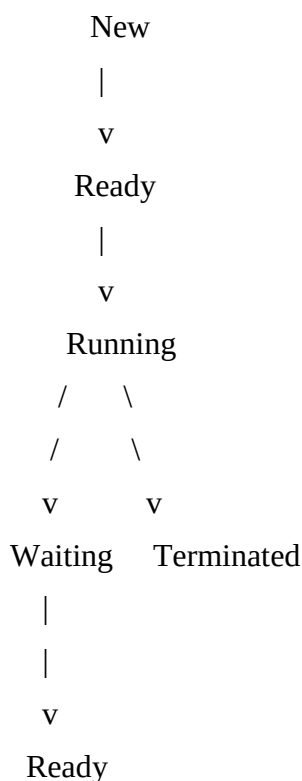
4. Waiting or Blocked State

Process किसी I/O Operation के पूरा होने की प्रतीक्षा कर रही होती है।

5. Terminated State

Process का कार्य पूरा हो चुका होता है।

Process State Diagram



Process Control Block (PCB)

Operating System प्रत्येक Process की जानकारी रखने के लिए एक विशेष डेटा संरचना का उपयोग करता है जिसे **Process Control Block (PCB)** कहा जाता है।

इसे Process का पहचान पत्र (Identity Card) भी कहा जाता है।

PCB में संग्रहित जानकारी

1. Process ID (PID)

प्रत्येक Process का एक अद्वितीय पहचान संख्या।

2. Process State

Process वर्तमान में किस अवस्था में है।

3. Program Counter

अगला निष्पादित होने वाला निर्देश।

4. CPU Registers

CPU Registers के मान।

5. CPU Scheduling Information

Priority, Queue Information आदि।

6. Memory Management Information

Base Address तथा Limit Register।

7. Accounting Information

CPU उपयोग समय आदि।

8. I/O Status Information

उपयोग किए जा रहे Devices की जानकारी।

PCB का महत्व

- Process Switching को संभव बनाता है।
- Context Switching में सहायता करता है।
- Scheduling के लिए आवश्यक जानकारी प्रदान करता है।

3. Process Scheduling

Process Scheduling क्या है?

जब कई Processes CPU के उपयोग के लिए प्रतीक्षा कर रही हों, तब यह निर्णय लेना कि CPU किस Process को दिया जाएगा, **Process Scheduling** कहलाता है।

Scheduler क्या है?

Scheduler वह प्रणाली है जो यह तय करती है कि अगली Process कौन-सी चलेगी।

Scheduler के प्रकार

1. Long-Term Scheduler

Processes को Ready Queue में भेजता है।

2. Short-Term Scheduler

Ready Queue से Process चुनकर CPU देता है।

3. Medium-Term Scheduler

Processes को Suspend और Resume करता है।

Scheduling Criteria

किसी Scheduling Algorithm की गुणवत्ता मापने के लिए निम्न मापदंड उपयोग किए जाते हैं:

1. CPU Utilization
2. Throughput
3. Turnaround Time
4. Waiting Time
5. Response Time

4. FCFS Scheduling (First Come First Serve)

परिचय

जिस Process का आगमन पहले होता है, उसे CPU पहले दिया जाता है।

यह Queue के सिद्धांत **FIFO (First In First Out)** पर आधारित है।

उदाहरण

Process	Burst Time
P1	5
P2	3
P3	2

क्रम:

P1 → P2 → P3

लाभ

- सरल और समझने में आसान।
- Implementation आसान।

हानियाँ

- Waiting Time अधिक हो सकता है।
- Convoy Effect उत्पन्न होता है।

5. SJF Scheduling (Shortest Job First)

परिचय

जिस Process का Burst Time सबसे कम होता है, उसे पहले CPU दिया जाता है।

उदाहरण

Process	Burst Time
P1	8
P2	4

P3	2
----	---

क्रम:

$P3 \rightarrow P2 \rightarrow P1$

लाभ

- Average Waiting Time सबसे कम होती है।

हानियाँ

- Burst Time पहले से ज्ञात होना चाहिए।
- Long Process को Starvation हो सकता है।

6. Priority Scheduling

परिचय

इस Scheduling में प्रत्येक Process को Priority दी जाती है।
सबसे अधिक Priority वाली Process पहले निष्पादित होती है।

उदाहरण

Process Priority

P1 3

P2 1

P3 2

क्रम:

$P2 \rightarrow P3 \rightarrow P1$

लाभ

- महत्वपूर्ण कार्यों को प्राथमिकता मिलती है।

हानियाँ

- कम Priority वाली Process को Starvation हो सकता है।

समाधान: Aging

लंबे समय से प्रतीक्षा कर रही Process की Priority बढ़ा दी जाती है।

7. Round Robin Scheduling

परिचय

इस Algorithm में प्रत्येक Process को निश्चित समय अवधि (Time Quantum) दी जाती है।
यदि Process निर्धारित समय में पूरी नहीं होती, तो उसे Queue के अंत में भेज दिया जाता है।

उदाहरण

Time Quantum = 2 ms

$P1 \rightarrow P2 \rightarrow P3 \rightarrow P1 \rightarrow P2$

लाभ

- Time Sharing Systems के लिए उपयुक्त।
- Response Time कम।

हानियाँ

- बहुत छोटा Time Quantum Context Switching बढ़ा देता है।
- बहुत बड़ा Time Quantum FCFS जैसा व्यवहार करता है।

8. Scheduling Algorithms

Scheduling Algorithms का उद्देश्य

- CPU Utilization बढ़ाना।
- Waiting Time कम करना।
- Throughput बढ़ाना।
- Fairness बनाए रखना।

मुख्य Scheduling Algorithms

1. FCFS
2. SJF
3. Priority Scheduling
4. Round Robin
5. Multilevel Queue Scheduling

9. Multiple Level Queue Scheduling

परिचय

इस Scheduling में Ready Queue को कई अलग-अलग Queues में विभाजित किया जाता है।

उदाहरण:

System Processes

Interactive Processes

Batch Processes

Student Processes

प्रत्येक Queue के लिए अलग Scheduling Algorithm हो सकती है।

उदाहरण

Queue	Algorithm
System Queue	Round Robin
Interactive Queue	Priority
Batch Queue	FCFS

लाभ

- विभिन्न प्रकार की Processes को अलग-अलग संभाला जा सकता है।

हानियाँ

- Queue के बीच Starvation हो सकती है।

10. Thread Concepts

Thread क्या है?

Thread किसी Process की सबसे छोटी निष्पादन इकाई (Smallest Unit of Execution) होती है।

इसे Lightweight Process भी कहा जाता है।

उदाहरण

यदि Web Browser एक Process है, तो उसके भीतर:

- Tab Handling
- Downloading
- Rendering

अलग-अलग Threads हो सकते हैं।

Thread के लाभ

- तेज Execution
- बेहतर Resource Sharing
- कम Memory उपयोग

Thread के प्रकार

1. User Level Thread

User Space में प्रबंधित।

2. Kernel Level Thread

Operating System द्वारा प्रबंधित।

Multithreading

एक Process के भीतर अनेक Threads का निष्पादन **Multithreading** कहलाता है।

11. Critical Section Problem

Critical Section क्या है?

किसी Program का वह भाग जहाँ Shared Resource का उपयोग किया जाता है, Critical Section कहलाता है।

उदाहरण

यदि दो Processes एक ही Bank Account Balance को एक साथ अपडेट करें, तो गलत परिणाम प्राप्त हो सकते हैं।

Critical Section की संरचना

Entry Section

Critical Section

Exit Section

Remainder Section

Critical Section Problem की शर्तें

1. Mutual Exclusion

एक समय में केवल एक Process Critical Section में प्रवेश कर सके।

2. Progress

यदि कोई Process Critical Section में नहीं है, तो किसी Process को प्रवेश करने से रोका नहीं जाना चाहिए।

3. Bounded Waiting

किसी Process को अनंत समय तक प्रतीक्षा नहीं करनी चाहिए।

12. Mutual Exclusion

Mutual Exclusion का अर्थ है:

एक समय में केवल एक Process Shared Resource का उपयोग कर सकती है।

यह Data Consistency बनाए रखने के लिए आवश्यक है।

Mutual Exclusion प्राप्त करने की विधियाँ

- Locks
- Semaphores
- Monitors
- Mutex

13. Semaphores

Semaphore क्या है?

Semaphore एक Synchronization Tool है जिसका उपयोग Processes के बीच समन्वय स्थापित करने और Shared Resources की सुरक्षा के लिए किया जाता है।

इसे सबसे पहले Edsger W. Dijkstra ने प्रस्तुत किया था।

Semaphore के मुख्य Operations

wait() या P()

यदि Resource उपलब्ध नहीं है, तो Process प्रतीक्षा करती है।

signal() या V()

Resource मुक्त होने पर अन्य Process को प्रवेश दिया जाता है।

Semaphore के प्रकार

1. Binary Semaphore

इसके केवल दो मान होते हैं:

0 और 1

यह Mutual Exclusion के लिए उपयोग किया जाता है।

2. Counting Semaphore

यह कई समान संसाधनों के प्रबंधन के लिए उपयोग किया जाता है।

उदाहरण:

यदि 5 Printers उपलब्ध हैं:

Semaphore = 5

Semaphore के लाभ

- Race Condition रोकता है।
- Synchronization प्रदान करता है।

- Shared Resources की सुरक्षा करता है।

Semaphore की हानियाँ

- Programming जटिल हो सकती है।
- Deadlock की संभावना रहती है।

UNIT – II का सारांश

इस यूनिट में हमने निम्न विषयों का अध्ययन किया:

1. Process की अवधारणा
2. Process States और PCB
3. Process Scheduling
4. FCFS, SJF, Priority एवं Round Robin Algorithms
5. Multiple Level Queue Scheduling
6. Threads और Multithreading
7. Critical Section Problem
8. Mutual Exclusion
9. Semaphores

महत्वपूर्ण परीक्षा प्रश्न

लघु उत्तरीय प्रश्न

1. Process क्या है?
2. PCB क्या है?
3. Thread क्या है?
4. Semaphore क्या है?
5. Mutual Exclusion क्या है?

दीर्घ उत्तरीय प्रश्न

1. Process States को चित्र सहित समझाइए।
2. Process Control Block का विस्तृत वर्णन कीजिए।
3. FCFS और SJF Scheduling की तुलना कीजिए।
4. Round Robin Scheduling को उदाहरण सहित समझाइए।
5. Critical Section Problem तथा उसकी शर्तों का वर्णन कीजिए।
6. Semaphore के प्रकार एवं कार्यप्रणाली को समझाइए।

UNIT – III

Process Synchronization, Deadlock और Memory Management

इस यूनिट में हम ऑपरेटिंग सिस्टम की उन महत्वपूर्ण समस्याओं और तकनीकों का अध्ययन करेंगे जो कंप्यूटर सिस्टम में कई Processes के एक साथ कार्य करने के दौरान उत्पन्न होती हैं। जब अनेक Processes एक ही संसाधन का उपयोग करना चाहती हैं, तब Synchronization की आवश्यकता होती है। इसी प्रकार संसाधनों के गलत आवंटन के कारण Deadlock जैसी समस्या उत्पन्न होती है। इसके अतिरिक्त Memory Management भी Operating System का एक महत्वपूर्ण कार्य है।

1. Classical Problems of Synchronization

(Synchronization की पारंपरिक समस्याएँ)

जब कई Processes या Threads एक ही समय पर साझा संसाधनों (Shared Resources) का उपयोग करते हैं, तब Data की शुद्धता बनाए रखने के लिए Synchronization की आवश्यकता होती है।

Operating System में Synchronization को समझाने के लिए कुछ प्रसिद्ध समस्याओं का उपयोग किया जाता है जिन्हें **Classical Synchronization Problems** कहा जाता है।

1.1 Producer-Consumer Problem

इसे **Bounded Buffer Problem** भी कहा जाता है।

समस्या का परिचय

इस समस्या में दो प्रकार की Processes होती हैं:

Producer

यह डेटा या वस्तुओं का निर्माण करता है।

Consumer

यह Producer द्वारा निर्मित डेटा का उपयोग करता है।

दोनों एक साझा Buffer का उपयोग करते हैं।

उदाहरण

मान लीजिए एक प्रिंटिंग सिस्टम में:

- Producer = User Process
- Consumer = Printer

यदि Producer बहुत तेजी से डेटा उत्पन्न करे और Buffer भर जाए, तो समस्या उत्पन्न होगी।

इसी प्रकार यदि Consumer डेटा माँगे और Buffer खाली हो, तब भी समस्या होगी।

समाधान

Semaphore का उपयोग करके:

- Empty Semaphore → खाली स्थानों की संख्या।
- Full Semaphore → भरे हुए स्थानों की संख्या।
- Mutex Semaphore → Mutual Exclusion प्रदान करता है।

Producer कार्य

wait(empty)

wait(mutex)

Insert Item

signal(mutex)

signal(full)

Consumer कार्य

wait(full)

wait(mutex)

Remove Item

signal(mutex)

signal(empty)

1.2 Readers-Writers Problem

परिचय

इस समस्या में दो प्रकार की Processes होती हैं:

1. Readers
2. Writers

Readers केवल डेटा पढ़ते हैं जबकि Writers डेटा को संशोधित करते हैं।

नियम

- एक समय में अनेक Readers डेटा पढ़ सकते हैं।
- लेकिन जब Writer लिख रहा हो, तब कोई Reader या दूसरा Writer प्रवेश नहीं कर सकता।

उदाहरण

Library Management System:

- विद्यार्थी पुस्तक पढ़ सकते हैं।
- लेकिन पुस्तक में परिवर्तन केवल Librarian कर सकता है।

समस्या

यदि लगातार Readers आते रहें, तो Writer को बहुत देर तक प्रतीक्षा करनी पड़ सकती है।

इसे **Writer Starvation** कहते हैं।

1.3 Dining Philosophers Problem

यह Synchronization की सबसे प्रसिद्ध समस्या है।

समस्या का विवरण

पाँच दार्शनिक (Philosophers) गोल मेज पर बैठे हैं।

प्रत्येक दार्शनिक:

- सोचता है (Thinking)
- भोजन करता है (Eating)

भोजन करने के लिए दो Forks की आवश्यकता होती है।

प्रत्येक दार्शनिक के पास केवल बायाँ और दायाँ Fork उपलब्ध है।

समस्या

यदि सभी दार्शनिक एक साथ बायाँ Fork उठा लें, तो कोई भी दायाँ Fork प्राप्त नहीं कर पाएगा।

परिणामस्वरूप सभी अनंत समय तक प्रतीक्षा करेंगे।

यह स्थिति Deadlock कहलाती है।

समाधान

- Semaphore का उपयोग
- Resource Ordering
- Maximum 4 Philosophers को अनुमति देना

2. Deadlock (डेडलॉक)

Deadlock क्या है?

जब दो या दो से अधिक Processes एक-दूसरे द्वारा उपयोग किए जा रहे संसाधनों की प्रतीक्षा करती रहती हैं और कोई भी Process आगे नहीं बढ़ पाती, तो इस स्थिति को **Deadlock** कहते हैं।

उदाहरण

मान लीजिए:

- Process P1 के पास Printer है और उसे Scanner चाहिए।
- Process P2 के पास Scanner है और उसे Printer चाहिए।

दोनों एक-दूसरे की प्रतीक्षा करेंगी और कभी समाप्त नहीं होंगी।

Deadlock का चित्र

P1 → Scanner

↑ ↓

Printer ← P2

Deadlock होने की आवश्यक शर्तें

Coffman Conditions

Deadlock होने के लिए निम्न चार शर्तों का एक साथ उपस्थित होना आवश्यक है।

1. Mutual Exclusion

संसाधन एक समय में केवल एक Process को उपलब्ध होगा।

उदाहरण:

Printer।

2. Hold and Wait

Process एक संसाधन को पकड़कर दूसरे संसाधन की प्रतीक्षा करती है।

3. No Preemption

किसी Process से संसाधन बलपूर्वक वापस नहीं लिया जा सकता।

4. Circular Wait

Processes एक चक्र बनाकर एक-दूसरे की प्रतीक्षा करती हैं।

2.1 Deadlock Prevention

(डेडलॉक की रोकथाम)

इस विधि में Deadlock की आवश्यक शर्तों में से किसी एक को समाप्त कर दिया जाता है।

Mutual Exclusion हटाना

यदि संभव हो तो संसाधनों को साझा बनाया जाए।

Hold and Wait हटाना

Process को सभी संसाधन एक साथ माँगने होंगे।

No Preemption हटाना

जरूरत पड़ने पर संसाधन वापस ले लिए जाएँ।

Circular Wait हटाना

संसाधनों को एक निश्चित क्रम में आवंटित किया जाए।

लाभ

- Deadlock कभी नहीं होगा।

हानि

- संसाधनों का उपयोग कम प्रभावी हो सकता है।

2.2 Deadlock Avoidance

(डेडलॉक से बचाव)

इस तकनीक में Operating System पहले से जाँचता है कि संसाधन देने के बाद सिस्टम सुरक्षित स्थिति (Safe State) में रहेगा या नहीं।

यदि सिस्टम Unsafe State में जाने वाला हो, तो संसाधन आवंटित नहीं किया जाता।

Safe State

ऐसी स्थिति जिसमें सभी Processes अंततः अपना कार्य पूरा कर सकती हैं।

Unsafe State

ऐसी स्थिति जिसमें भविष्य में Deadlock की संभावना हो।

Banker's Algorithm

Deadlock Avoidance के लिए सबसे प्रसिद्ध Algorithm है।

इसे Edsger W. Dijkstra ने विकसित किया था।

Banker's Algorithm का सिद्धांत

जैसे बैंक यह सुनिश्चित करता है कि उसके पास सभी ग्राहकों की माँग पूरी करने के लिए पर्याप्त धन हो, उसी प्रकार Operating System यह सुनिश्चित करता है कि उसके पास पर्याप्त संसाधन उपलब्ध हों।

2.3 Deadlock Detection

(डेडलॉक का पता लगाना)

इस तकनीक में Operating System Deadlock को होने देता है और बाद में उसकी पहचान करता है।

Detection Methods

1. Wait-for Graph

यदि ग्राफ में Cycle बनती है, तो Deadlock मौजूद है।

2. Resource Allocation Graph

Processes और Resources के बीच संबंध दर्शाता है।

2.4 Deadlock Recovery

(डेडलॉक से उबरना)

Deadlock का पता चलने के बाद उसे समाप्त करने के लिए Recovery Techniques का उपयोग किया जाता है।

1. Process Termination

एक या अधिक Processes को समाप्त कर दिया जाता है।

2. Resource Preemption

Processes से संसाधन वापस लेकर अन्य Processes को दिए जाते हैं।

3. Rollback

Process को पूर्व सुरक्षित स्थिति में वापस भेजा जाता है।

3. Memory Management

(स्मृति प्रबंधन)

Memory Management Operating System का वह कार्य है जिसके अंतर्गत Main Memory का प्रबंधन किया जाता है।

मुख्य कार्य

1. Memory Allocation
2. Memory Deallocation
3. Memory Protection
4. Memory Sharing

Memory Management की आवश्यकता

- अनेक Processes को Memory प्रदान करना।
- Memory का सर्वोत्तम उपयोग करना।
- सुरक्षा सुनिश्चित करना।

4. Logical Address Space और Physical Address Space

Logical Address

CPU द्वारा उत्पन्न Address को Logical Address कहते हैं।

इसे Virtual Address भी कहा जाता है।

Physical Address

Memory Unit में वास्तविक स्थान को Physical Address कहते हैं।

उदाहरण

मान लीजिए:

Logical Address = 1200

Memory Management Unit (MMU) इसे परिवर्तित करके:

Physical Address = 5200

अंतर

Logical Address	Physical Address
CPU द्वारा उत्पन्न	Memory द्वारा उपयोग

User को दिखाई देता है	User को दिखाई नहीं देता
Virtual Address कहलाता है	Real Address कहलाता है

Memory Management Unit (MMU)

यह Logical Address को Physical Address में परिवर्तित करती है।

5. Swapping

Swapping क्या है?

जब Main Memory में स्थान कम हो जाता है, तब Operating System किसी Process को अस्थायी रूप से Secondary Storage में भेज देता है और आवश्यकता होने पर वापस Memory में लाता है।

Swapping की प्रक्रिया

Main Memory



Swap Out



Hard Disk



Swap In



Main Memory

लाभ

- अधिक Processes को चलाने की सुविधा।
- Memory का बेहतर उपयोग।

हानियाँ

- Disk Access धीमा होता है।
- Performance कम हो सकती है।

6. Contiguous Allocation

परिचय

इस विधि में Process को Memory के लगातार (Continuous) भाग में स्थान दिया जाता है।

उदाहरण

1000 - 2000 → Process P1

2001 - 3500 → Process P2

3501 - 4500 → Process P3

लाभ

- सरल Implementation।
- तेज Access।

हानियाँ

External Fragmentation

खाली स्थान छोटे-छोटे टुकड़ों में बँट जाता है।

Internal Fragmentation

आवृत्ति Memory का कुछ भाग उपयोग नहीं होता।

7. Non-Contiguous Allocation

परिचय

इस तकनीक में Process को Memory के अलग-अलग भागों में रखा जाता है।

प्रकार

1. Paging

Memory को निश्चित आकार के Frames में विभाजित किया जाता है।

Process को Pages में विभाजित किया जाता है।

उदाहरण

Page 1 → Frame 4

Page 2 → Frame 1

Page 3 → Frame 7

लाभ

- External Fragmentation समाप्त हो जाती है।

2. Segmentation

Program को Logical Segments में विभाजित किया जाता है।

उदाहरण:

- Code Segment
- Data Segment
- Stack Segment

लाभ

- Program Structure के अनुसार Memory Allocation।

Paging और Segmentation में अंतर

Paging	Segmentation
Fixed Size Blocks	Variable Size Blocks
Programmer को दिखाई नहीं देता	Programmer को दिखाई देता
Internal Fragmentation होती है	External Fragmentation होती है

UNIT – IV

Virtual Memory Management

इस यूनिट में हम Operating System की Memory Management से संबंधित उन्नत तकनीकों का अध्ययन करेंगे। आधुनिक कंप्यूटर सिस्टम में एक साथ कई प्रोग्राम चलाए जाते हैं, जिसके कारण RAM की आवश्यकता बढ़ जाती है। इस समस्या के समाधान के लिए Operating System Paging, Segmentation तथा Virtual Memory जैसी तकनीकों का उपयोग करता है।

1. Paging (पेजिंग)

Paging क्या है?

Paging एक **Memory Management Technique** है जिसमें Physical Memory को निश्चित आकार के भागों में तथा Process को समान आकार के भागों में विभाजित किया जाता है।

- Process के भागों को **Pages** कहा जाता है।
- Main Memory के भागों को **Frames** कहा जाता है।

Operating System प्रत्येक Page को किसी भी खाली Frame में रख सकता है।

उदाहरण

मान लीजिए:

- Page Size = 4 KB
- Process Size = 16 KB

तब Process को 4 Pages में विभाजित किया जाएगा।

Process P1

Page 0

Page 1

Page 2

Page 3

Main Memory

Frame 0

Frame 1

Frame 2

Frame 3

Frame 4

Frame 5

Page Table

Operating System प्रत्येक Process के लिए एक **Page Table** बनाए रखता है।

Page Table यह बताती है कि कौन-सा Page किस Frame में रखा गया है।

Page Number	Frame Number
0	3
1	7
2	2
3	5

Address Translation

Logical Address दो भागों से मिलकर बनता है:

1. Page Number
2. Offset

MMU (Memory Management Unit) Page Number की सहायता से Frame Number खोजती है और Physical Address तैयार करती है।

Paging के लाभ

- External Fragmentation समाप्त हो जाती है।
- Memory Utilization बढ़ता है।
- Virtual Memory संभव होती है।
- Multiprogramming को सहायता मिलती है।

Paging की हानियाँ

- Internal Fragmentation हो सकती है।
- Page Table के लिए अतिरिक्त Memory चाहिए।
- Address Translation में समय लगता है।

2. Segmentation (सेगमेंटेशन)

Segmentation क्या है?

Segmentation ऐसी तकनीक है जिसमें Program को उसके Logical Units के आधार पर विभाजित किया जाता है।

उदाहरण:

- Main Program
- Functions
- Stack
- Data Segment

Segmentation का उद्देश्य

प्रोग्रामर के दृष्टिकोण से Memory को व्यवस्थित करना।

उदाहरण

Program

Code Segment

Data Segment
Stack Segment
Heap Segment

Segment Table

Segmentation में Operating System प्रत्येक Segment के लिए:

- Base Address
- Limit

संग्रहित करता है।

Address Format

Logical Address:

<Segment Number, Offset>

Segmentation के लाभ

- Program Structure स्पष्ट रहती है।
- Memory Sharing आसान होती है।
- Protection लागू करना सरल होता है।

Segmentation की हानियाँ

- External Fragmentation उत्पन्न होती है।
- Memory Management जटिल होती है।

Paging और Segmentation में अंतर

Paging	Segmentation
Fixed Size Blocks	Variable Size Blocks
Programmer को दिखाई नहीं देता	Programmer को दिखाई देता
Internal Fragmentation	External Fragmentation
Hardware आधारित	Logical आधारित

3. Demand Paging (डिमांड पेजिंग)

Demand Paging क्या है?

Demand Paging एक ऐसी तकनीक है जिसमें Process के सभी Pages को एक साथ Memory में लोड नहीं किया जाता।

केवल वही Page Memory में लाया जाता है जिसकी वर्तमान समय में आवश्यकता होती है।

उदाहरण

यदि किसी Program में 100 Pages हैं लेकिन वर्तमान में केवल 10 Pages की आवश्यकता है, तो केवल वही 10 Pages RAM में लाए जाएंगे।

लाभ

- Memory की बचत होती है।
- अधिक Processes को चलाया जा सकता है।
- System Performance बढ़ती है।

हानि

यदि आवश्यक Page Memory में उपलब्ध नहीं होता, तो **Page Fault** उत्पन्न होता है।

Page Fault

जब CPU किसी ऐसे Page को Access करने का प्रयास करता है जो Memory में मौजूद नहीं है, तब Page Fault उत्पन्न होती है।

Page Fault Handling

1. CPU Page को खोजता है।
2. Page Memory में नहीं मिलता।
3. Interrupt उत्पन्न होता है।
4. आवश्यक Page Disk से Memory में लाया जाता है।
5. Process पुनः प्रारंभ होती है।

4. Page Replacement Algorithms

जब Main Memory पूरी तरह भर जाती है और नया Page लाना होता है, तब किसी पुराने Page को हटाना पड़ता है।

यह कार्य Page Replacement Algorithm द्वारा किया जाता है।

4.1 FIFO (First In First Out)

सबसे पहले Memory में आया हुआ Page सबसे पहले हटाया जाता है।

उदाहरण

Reference String:

1,2,3,4,1,2,5,1,2,3,4,5

यदि Memory में केवल 3 Frames उपलब्ध हैं:

सबसे पुराना Page हटाया जाएगा।

लाभ

- सरल Algorithm
- Implementation आसान

हानियाँ

- अधिक Page Fault हो सकते हैं।
- Belady's Anomaly उत्पन्न हो सकती है।

Belady's Anomaly

कुछ परिस्थितियों में Frames की संख्या बढ़ाने पर भी Page Fault बढ़ जाते हैं।

4.2 LRU (Least Recently Used)

जिस Page का सबसे लंबे समय से उपयोग नहीं हुआ है, उसे हटाया जाता है।

उदाहरण

यदि Pages:

1 2 3 4 2 1 5

और Memory पूर्ण है, तो सबसे कम उपयोग किए गए Page को हटाया जाएगा।

लाभ

- FIFO की तुलना में बेहतर।
- Page Fault कम होते हैं।

हानियाँ

- Implementation जटिल।
- अतिरिक्त Hardware Support की आवश्यकता।

4.3 LIFO (Last In First Out)

सबसे बाद में Memory में आए Page को सबसे पहले हटाया जाता है।

लाभ

- Implementation सरल।

हानियाँ

- Performance खराब हो सकती है।
- व्यावहारिक रूप से कम उपयोग किया जाता है।

5. Hit Ratio (हिट अनुपात)

Hit क्या है?

यदि CPU द्वारा माँगा गया Page पहले से ही Memory में मौजूद हो, तो उसे **Hit** कहते हैं।

Hit Ratio का सूत्र

$$\left[\text{Hit Ratio} = \frac{\text{Number of Hits}}{\text{Total Memory References}} \right]$$

उदाहरण

यदि:

- Total References = 100
- Hits = 85

तब:

$$\left[\text{Hit Ratio} = \frac{85}{100} = 0.85 \right]$$

अर्थात:

$$\text{Hit Ratio} = 85\%$$

Miss Ratio

यदि Page Memory में उपलब्ध नहीं है:

[
Miss\ Ratio = 1 - Hit\ Ratio
]

6. Virtual Memory Concepts

Virtual Memory क्या है?

Virtual Memory ऐसी तकनीक है जो Secondary Storage को Main Memory के विस्तार के रूप में उपयोग करती है।

आवश्यकता

- RAM सीमित होती है।
- बड़े Programs चलाने होते हैं।
- Multiprogramming बढ़ानी होती है।

कार्यप्रणाली

CPU

↓

RAM

↓

Virtual Memory

↓

Hard Disk

लाभ

- बड़े Program चल सकते हैं।
- Multiprogramming बढ़ती है।
- Memory Utilization बेहतर होता है।

हानियाँ

- Page Fault बढ़ने पर Performance कम होती है।
- Disk Access धीमा होता है।

UNIT – V

1. Disk Scheduling Algorithms

परिचय

जब कई Processes एक साथ Disk Access का अनुरोध करती हैं, तब Operating System यह निर्णय लेता है कि Disk Head पहले किस Request को पूरा करेगी।

इस प्रक्रिया को Disk Scheduling कहते हैं।

Disk Scheduling के उद्देश्य

- Seek Time कम करना।
- Response Time कम करना।
- Throughput बढ़ाना।

1.1 FCFS Disk Scheduling

जिस Request का आगमन पहले होता है, उसे पहले पूरा किया जाता है।

लाभ:

- सरल।

हानि:

- Seek Time अधिक।

1.2 SSTF (Shortest Seek Time First)

जिस Request तक पहुँचने में सबसे कम दूरी तय करनी पड़े, उसे पहले पूरा किया जाता है।

लाभ:

- Seek Time कम।

हानि:

- Starvation संभव।

1.3 SCAN Algorithm

Disk Head एक दिशा में चलती है और रास्ते में आने वाली सभी Requests पूरी करती है।

इसे Elevator Algorithm भी कहा जाता है।

1.4 C-SCAN Algorithm

Head केवल एक दिशा में चलती है और अंत में वापस प्रारंभिक स्थिति पर लौटती है।

1.5 LOOK Algorithm

SCAN की तरह कार्य करता है लेकिन Disk के अंत तक नहीं जाता।

1.6 C-LOOK Algorithm

LOOK का Circular संस्करण।

2. Swap Space Management

Swap Space क्या है?

Hard Disk का वह भाग जिसे Operating System Virtual Memory के लिए उपयोग करता है।

उपयोग

- RAM भर जाने पर।
- Inactive Processes को संग्रहीत करने के लिए।

लाभ

- अधिक Processes चल सकती हैं।

हानियाँ

- Disk Access धीमा होता है।

3. Disk Reliability

Disk Reliability क्या है?

Disk Reliability का अर्थ है डेटा को सुरक्षित और विश्वसनीय बनाए रखना।

डेटा हानि के कारण

- Power Failure
- Hardware Failure
- Virus Attack
- Bad Sectors

समाधान

- Backup
- RAID Technology
- Error Detection

RAID (Redundant Array of Independent Disks)

यह कई Disks को मिलाकर Reliability बढ़ाने की तकनीक है।

4. Stable Storage Implementation

Stable Storage क्या है?

ऐसी Storage जो Hardware Failure होने पर भी डेटा को सुरक्षित रखे।

तकनीक

- Data Replication
- Mirroring
- RAID

5. Types of Mobile Operating Systems

प्रमुख Mobile Operating Systems

- Android
- iOS
- Windows Phone
- BlackBerry OS
- Symbian

6. Overview of Android OS

Android क्या है?

Android एक Open Source Mobile Operating System है जिसे मुख्य रूप से Smartphones और Tablets के लिए विकसित किया गया है।

यह मुख्य रूप से Google द्वारा विकसित किया जाता है।

Android Architecture

Applications

Application Framework

Libraries

Android Runtime

Linux Kernel

Android की विशेषताएँ

- Open Source
- Multitasking
- Touch Interface
- Application Support
- Security Features

Android के उपयोग

- Smartphones
- Smart TV
- Smart Watch
- Car Systems

7. Applications of Mobile Operating Systems

1. Communication
2. Internet Browsing
3. Banking
4. Education
5. Entertainment
6. Navigation
7. E-Commerce
8. Health Monitoring

8. Overview of Linux OS

Linux क्या है?

Linux एक Open Source Operating System है जिसका विकास Linus Torvalds ने किया था।

Linux की विशेषताएँ

- Open Source
- Multiuser
- Multitasking
- Secure
- Portable

Linux के प्रमुख Distribution

- Ubuntu
- Fedora
- Debian
- Red Hat Enterprise Linux

Linux के उपयोग

- Servers
- Cloud Computing